

Title: CIS System — Overview for Stakeholders

Notes: 5-minute briefing; 12 slides ~25s each.

Agenda

- System summary
- Architecture and dataflow
- Key components: detectors, models, intervention
- Deployment, testing, operations
- Recommendations

Notes: Briefly list what you'll cover and time goal.

System Summary

- Purpose: detect and respond to host-level anomalies and ransomware
- Components: sensors (eBPF), detectors, models, intervention engine, dashboard
- Deployment: containers, systemd, Windows installer

Notes: One-sentence elevator pitch + core components.

Architecture & Components

- `cis/`: backend: main_detector, immune_memory, intervention_engine
- `ebpf/`: kernel-level sensors
- `dashboard/`: UI and visualization
- `deploy/` and `installer/`: deployment artifacts

Notes: High-level diagram on slide; mention data stores and flows.

Detector Pipeline

- Sensor ingestion → preprocessing → model inference → alert decision
- `immune\_memory` reduces false positives
- Policies in `alert\_rules.json` govern actions

Notes: Explain how events become alerts and role of immune memory.

eBPF & Host Integration

- eBPF programs collect syscall/file activity (in `ebpf`)
- Loader (`cis-ebpf-loader` / side-channel) inserts BPF programs safely
- Advantages: low-overhead, kernel-level visibility

Notes: Emphasize kernel visibility and compatibility testing.

Models & Training

- ``cis/train.py`` and ``cis/model.py`` produce inference artifacts
- Models stored in ``models/`` and versioned
- Retraining workflow available in repo

Notes: Note data needs and model validation steps.

Intervention Engine & Dashboard

- ``intervention_engine.py`` maps alerts to actions (notify/contain)
- Side-channel daemon executes host actions and health checks
- Dashboard shows alerts and acknowledgment flows

Notes: Show example mitigation scenarios and dashboard screenshot.

Deployment Options

- Containers: `deploy/container/docker-compose.yml`
- Systemd units: `deploy/systemd/` for Linux production
- Windows: `installer/` scripts and `build\_installer\_windows.bat`

Notes: Recommend containerized dev; systemd for production.

Testing & Verification

- Unit tests in `cis/tests/` (pytest)
- Smoke tests: `deploy/scripts/smoke\_test.sh`
- eBPF validation: kernel compatibility and loader verification

Notes: Mention CI to run tests and image builds.

Operational Considerations & Security

- Logging, metrics export, and rotation
- Secrets management (avoid plaintext in ``cis/config.json``)
- Container image scanning and eBPF safety checks

Notes: Call out monitoring & backup of model/artifacts.

Recommendations & Next Steps

- Add CI (tests, lint, build images)
- Add Prometheus metrics and health endpoints
- Document installer and add architecture diagram to `docs/`
- Short roadmap: demo -> CI -> monitoring -> hardened deploy

Notes: Close with ask and contact for follow-up.